Wine quality prediction

import important libraries

In [1]:

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

loading dataset of WineQT

In [2]:

```python
df=pd.read_csv("WineQT.csv")
```

Check first 5 rows

In [3]:

```python
df.head()
```

Out[3]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide
0	7.4	0.70	0.00	1.9	0.076	11.0

1	7.8	0.88	0.00	2.6	0.098	25.0
2	7.8	0.76	0.04	2.3	0.092	15.0
3	11.2	0.28	0.56	1.9	0.075	17.0
4	7.4	0.70	0.00	1.9	0.076	11.0

In [4]:

```
df.drop('Id',axis=1,inplace=True)
```

Unique Quality

In [5]:

```
df['quality'].unique()
```

Out[5]:

```
array([5, 6, 7, 4, 8, 3], dtype=int64)
```

Checking for null values

In [6]:

```
df.isnull().sum()
```

Out[6]:

```
fixed acidity           0
volatile acidity        0
citric acid             0
residual sugar          0
chlorides               0
free sulfur dioxide     0
total sulfur dioxide    0
density                 0
pH                      0
sulphates               0
alcohol                 0
```

```
quality                 0
dtype: int64
```

Descriptive Statistics

In [7]:

```
df.describe()
```

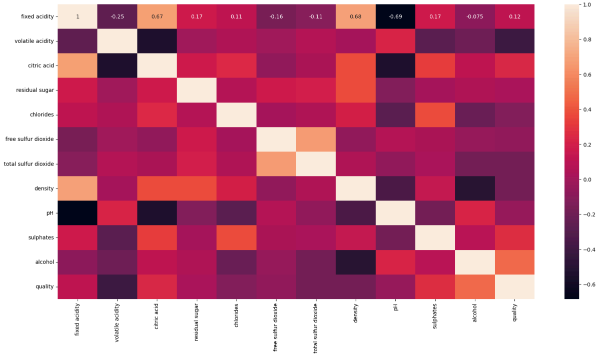
Out[7]:

	fixed acidity	volatile acidity	citric acid	
count	1143.000000	1143.000000	1143.000000	114
mean	8.311111	0.531339	0.268364	
std	1.747595	0.179633	0.196686	
min	4.600000	0.120000	0.000000	
25%	7.100000	0.392500	0.090000	
50%	7.900000	0.520000	0.250000	
75%	9.100000	0.640000	0.420000	
max	15.900000	1.580000	1.000000	1

Correlation Matrix

In [8]:

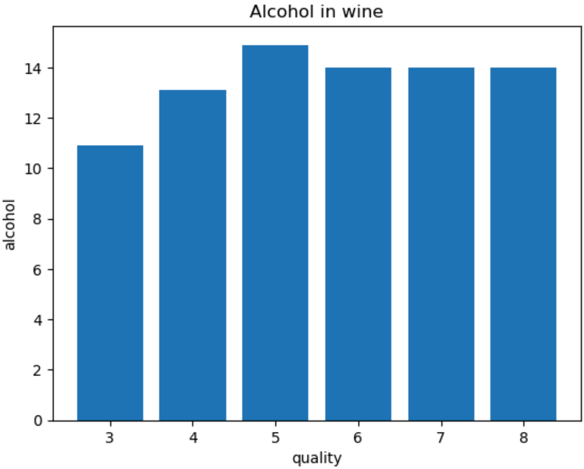
```
plt.figure(figsize=(20,10))
sns.heatmap(df.corr(),annot=True)
plt.show()
```



Visualisation

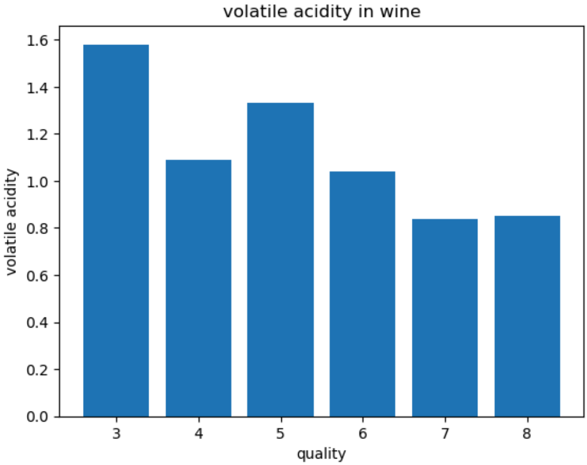
In [15]:

```
plt.bar(df['quality'],df['alcohol'],data=df
plt.xlabel("quality")
plt.ylabel("alcohol")
plt.title("Alcohol in wine")
plt.show()
```



In [16]:

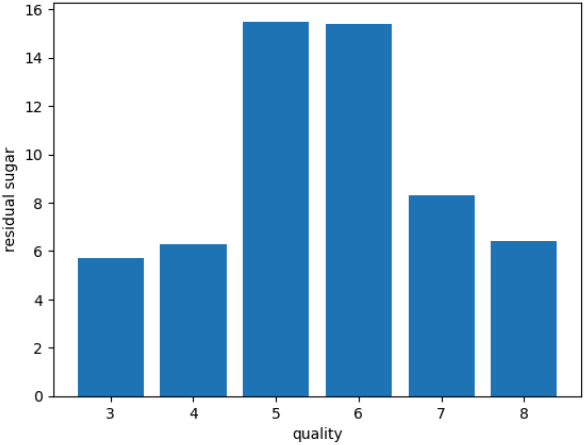
```
plt.bar(df['quality'],df['volatile acidity'
plt.xlabel("quality")
plt.ylabel("volatile acidity")
plt.title("volatile acidity in wine")
plt.show()
```



In [17]:

```
plt.bar(df['quality'],df['residual sugar'],
plt.xlabel("quality")
plt.ylabel("residual sugar")
plt.title("residual sugar in wine")
plt.show()
```

residual sugar in wine



In [19]:

```
df['good_quality']=[1 if x>=6 else 0 for x
```

In [22]:

```
df.drop(['quality'],axis=1,inplace=True)
```

In [23]:

```
df.head()
```

Out[23]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide
0	7.4	0.70	0.00	1.9	0.076	11.0
1	7.8	0.88	0.00	2.6	0.098	25.0
2	7.8	0.76	0.04	2.3	0.092	15.0
3	11.2	0.28	0.56	1.9	0.075	17.0

4	7.4	0.70	0.00	1.9	0.076	11.0

In [75]:

```
df['good_quality'].value_counts()   # proper
```

Out[75]:

```
good_quality
1    621
0    522
Name: count, dtype: int64
```

In [50]:

```
features=df.drop(['good_quality'],axis=1)
target=df['good_quality']
```

In [51]:

```
from sklearn.model_selection import train_t
```

In [64]:

```
x_train,x_test,y_train,y_test=train_test_sp
```

In [65]:

```
len(x_train)
```

Out[65]:

```
914
```

In [66]:

```
len(x_test)
```

Out[66]:

```
229
```

Model traning

Logistic Regression

In [67]:

```
from sklearn.linear_model import LogisticRe
lr=LogisticRegression()
lr.fit(x_train,y_train)
```

```
C:\ProgramData\anaconda3\Lib\site-packages
\sklearn\linear_model\_logistic.py:458: Con
vergenceWarning: lbfgs failed to converge
 (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMI
T.

Increase the number of iterations (max_ite
r) or scale the data as shown in:
    https://scikit-learn.org/stable/module
s/preprocessing.html
Please also refer to the documentation for
alternative solver options:
    https://scikit-learn.org/stable/module
s/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

Out[67]:

```
▼ LogisticRegression
LogisticRegression()
```

Now our model is ready to predict

In [71]:

```
x_test
```

Out[71]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	f su diox

567	7.4	0.370	0.43	2.6	0.082	1
268	8.3	0.260	0.42	2.0	0.080	1
220	7.6	0.620	0.32	2.2	0.082	
1016	7.6	0.410	0.33	2.5	0.078	
327	11.5	0.315	0.54	2.1	0.084	
...	...	...	...	...	...	
96	5.2	0.340	0.00	1.8	0.050	2
164	8.5	0.370	0.20	2.8	0.090	1
41	7.3	0.390	0.31	2.4	0.074	
308	11.1	0.450	0.73	3.2	0.066	
449	10.1	0.935	0.22	3.4	0.105	1

229 rows × 11 columns

In [72]:

```
lr.predict(x_test)
```

Out[72]:

```
array([0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0,
1, 1, 1, 1, 1, 1, 0, 1, 1, 0,
       1, 0, 1, 0, 1, 0, 0, 0, 1, 1, 0, 0,
0, 1, 0, 1, 0, 0, 0, 1, 1, 0,
       1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1,
1, 0, 1, 0, 1, 1, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 0,
1, 1, 1, 0, 1, 1, 1, 1, 0, 0,
       0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1,
```

```
0, 1, 1, 1, 1, 0, 0, 1, 1, 0,
       1, 1, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0,
0, 0, 0, 1, 1, 1, 1, 1, 0, 0,
       0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0,
0, 0, 0, 0, 0, 1, 0, 0, 1, 1,
       1, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0,
1, 1, 0, 1, 0, 1, 0, 1, 1, 0,
       1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0,
1, 1, 0, 0, 1, 0, 1, 0, 1, 1,
       1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0,
0, 1, 0, 0, 0, 1, 0, 1, 1, 1,
       1, 0, 1, 0, 1, 0, 0, 1, 0], dtype=in
t64)
```

Check model score

In [74]:

```
lr.score(x_test,y_test)
```

Out[74]:

```
0.6986899563318777
```

Random Forest

In [79]:

```
from sklearn.ensemble import RandomForestCl
```

In [88]:

```
clf=RandomForestClassifier(random_state=1)
```

In [89]:

```
clf.fit(x_train,y_train)
```

Out[89]:

```
▼         RandomForestClassifier
RandomForestClassifier(random_state=1)
```

In [90]:

```
clf.predict(x_test)
```

Out[90]:

```
array([0, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 0, 1, 1, 1,
       0, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1,
0, 1, 0, 1, 1, 1, 0, 1, 1, 1,
       1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1,
1, 0, 1, 0, 1, 1, 1, 0, 0, 0,
       0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0,
1, 1, 1, 0, 1, 1, 1, 0, 0, 0,
       0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1, 1,
0, 0, 1, 1, 1, 1, 0, 1, 1, 0,
       0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0,
0, 0, 0, 1, 1, 1, 1, 1, 1, 0,
       0, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0,
0, 0, 1, 0, 0, 1, 0, 0, 1, 1,
       1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 1, 1,
1, 0, 0, 1, 0, 1, 1, 1, 1, 0,
       1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 0,
1, 1, 1, 0, 1, 0, 1, 1, 1, 1,
       1, 1, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1,
0, 1, 0, 0, 0, 1, 0, 1, 1, 1,
       1, 0, 1, 0, 1, 0, 0, 1, 0], dtype=in
t64)
```

In [91]:

```
clf.score(x_test,y_test)
```

Out[91]:

```
0.777292576419214
```

Kneighbors Classifier

In [92]:

```
from sklearn.neighbors import KNeighborsCla
```

In [93]:

```
knn=KNeighborsClassifier(n_neighbors=3)
knn.fit(x_train,y_train)
```

Out[93]:

```
▼ KNeighborsClassifier
KNeighborsClassifier(n_neighbors=3)
```

In [94]:

```
knn.predict(x_test)
```

Out[94]:

```
array([0, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0,
1, 1, 1, 1, 1, 1, 0, 0, 1, 1,
       1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0,
1, 0, 0, 0, 1, 1, 0, 1, 1, 1,
       0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1,
1, 0, 0, 0, 1, 1, 1, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0,
0, 1, 0, 0, 1, 1, 1, 0, 0, 0,
       0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1,
0, 0, 1, 1, 1, 1, 0, 1, 1, 0,
       1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0,
0, 1, 0, 0, 1, 1, 1, 1, 1, 0,
       0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 0,
1, 1, 0, 0, 1, 1, 1, 0, 1, 1,
       1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0,
0, 0, 0, 1, 1, 0, 1, 1, 0, 0,
       1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0,
0, 0, 0, 1, 0, 1, 1, 0, 0, 1,
       1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1,
0, 0, 0, 0, 0, 1, 0, 0, 1, 1,
       1, 0, 0, 0, 1, 0, 0, 1, 0], dtype=in
t64)
```

In [95]:

```
knn.score(x_test,y_test)
```

Out[95]:

```
0.6375545851528385
```

In [ ]:

```
```